

Document Intelligence

Financial document extraction and validation

Python API · page-aware OCR · structured data · persistent results

Local build verified. Live AI extraction and public deployment remain outstanding.

Problem and requirements

Invoices and statements arrive as scanned images or native PDFs.

Extract visible fields and tables without inventing missing values.

Check financial relationships separately for each reporting period.

Make results available through a persistent dashboard and REST APIs.

Dataset inspection

50

supplied documents

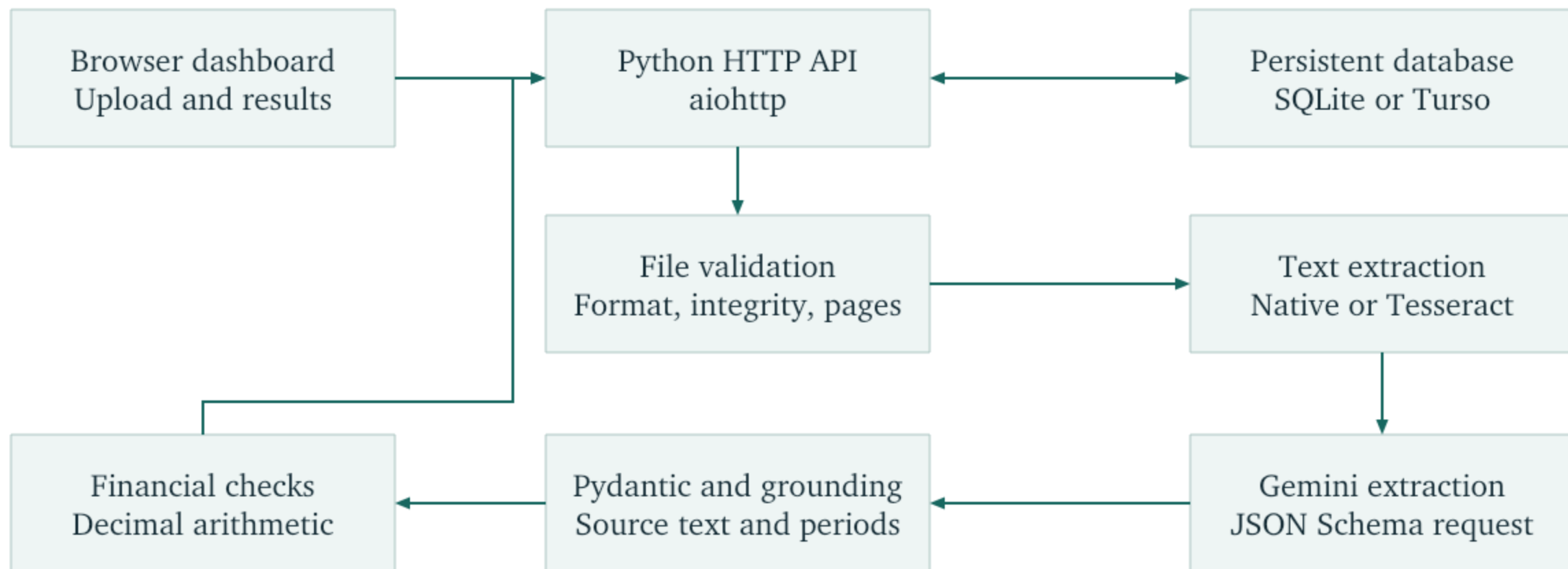
57

pages and images processed

20 invoice images and 10 examples of each financial statement type

29 of the 30 PDFs require OCR. One cash-flow PDF has native text.

Application architecture



Controlled errors and request-ID stage logs span the pipeline.

Input validation

Allow PDF, JPG and PNG only, with matching file signatures.

Reject empty, corrupted, encrypted, oversized and over-three-page files.

Sanitize filenames and bound image-rendering memory.

Return controlled JSON errors; retain processing failures where possible.

OCR and page boundaries

Native PDF page

Extract positioned words.
Preserve larger column gaps.
Skip OCR when the text is usable.

Scanned page or image

Apply EXIF orientation and contrast.
Run bounded Tesseract OCR.
Compare rotation candidates for weak text.

Every page retains its text, extraction method and page number.

Structured AI extraction

Pydantic defines the provider schema and the output contract.

Extensible fields and tables preserve additional visible information.

Comparative periods remain separate; original labels are retained.

Missing values are null. Malformed or incomplete responses fail clearly.

Live model accuracy and completeness still require evaluation with an API key.

Evidence and conservative normalization

Printed total

\$ 138,90

Accepted structured value

value: "138,90"

numeric_value: "138.90"

page_number: 1

A quote must exist on the claimed page and contain the exact raw value.
Grounding supports traceability; it cannot prove the OCR read the image correctly.

Deterministic financial validation

Invoice total

$$126.27 + 12.63 = 138.90$$

Cash-flow net change, 2022

$$\begin{aligned} &-11,959.57 - 2,216.33 + 48,124.02 + 165.10 \\ &= 34,113.22 \end{aligned}$$

Absolute tolerance: 0.05 in displayed units. Missing operands: NOT_APPLICABLE.

API and persistence

POST /api/v1/documents/process
GET /api/v1/documents/{document_name}
GET /api/v1/documents
GET /api/v1/health

One saved result per sanitized filename; reprocessing updates the latest record.

SQLite locally. External Turso storage for a host with an ephemeral disk.

Dashboard workflow

Upload and browse

Select the document type.
Upload one supported file.
Search saved results by filename.
Open the latest database record.

Inspect the result

Read fields and financial tables.
Expand source evidence.
Inspect checks and missing values.
Expand the full JSON response.

Same-origin requests connect the frontend to the deployed backend.

Executed verification

54 automated tests passed

Real OCR/native text recovery on all 50 supplied documents.

Four annotated reference replays exercised all document categories.

Standalone HTTP smoke checks retained four results after restart.

Browser, live model, live Turso and Docker build remain unverified.

Issues found and fixed

Incorrect receipt rotation

OSD suggested a 180° flip for upright receipts. The pipeline now compares recognition quality before accepting a rotation.

Encoded upload filenames

Multipart clients encoded spaces in filenames. Decoding before sanitization restored consistent GET-by-name behavior; a regression test covers it.

Deployment preparation

Docker packages Python and Tesseract in one service.

Render configuration reads PORT and serves frontend plus API.

Turso keeps records outside the free service's ephemeral filesystem.

Deploy, run a real upload, and verify persistence before submitting URLs.

Outstanding verification and limitations

Live Gemini response quality and completeness need an authorized API key.

Browser behavior, Docker build and live Turso connectivity need verification.

OCR errors remain possible in faint, skewed or densely printed documents.

Public GitHub and verified live application URLs are still required.

AI tools and production improvements

Tools used

Codex/ChatGPT for implementation, tests, debugging and documentation.
PyMuPDF and Tesseract for real dataset text recovery.

Next engineering steps

Add authentication and retention.
Measure a labelled evaluation set.
Add source-image review and correction.
Move longer work to background jobs.

Interview demonstration

Upload an invoice image and inspect its line items and evidence.

Process a comparative statement and explain period-specific checks.

Show a missing operand and a deliberate financial failure.

Restart the service, retrieve the saved result, and explain the architecture.

Be explicit about which checks ran live and which use test fixtures.